Reading the Structure of a Differential Equation from Data

A proposed learning pipeline built around an exact analytic core,
and two questions for you

Draft for discussion

October 27, 2026

What I would like to ask you (details in Part III)

Q1. The first step fits the data with a family of *basis functions*. Do I have to choose them in advance (and therefore only discover equations whose building blocks I anticipated), or is there a way to avoid that choice?

Q2. I built the *analytic* part of the method. I would like your help to design the *outer* machine-learning structure: the loop, how to estimate its parameters, which encoder and which clustering algorithm.

More generally: do you think this is worth working on together?

Contents

Part I — The idea in plain words **1**

Part II — A little mathematics **3**

0.1 Setting . . . 3
0.2 The analytic core (exact data) . . . 3
0.3 Statistical wrapper (proposal) . . . 5
0.4 The loop . . . 5
0.5 What is known and what is not . . . 6

Part III — Questions for you **6**

Part I — The idea in plain words

An orchestra

Imagine you record a piece of music, but you do not know which instruments are playing. You would like a short list: *"70 % violin, 30 % drums"*.

A differential equation is like a piece of music, and a function $u(t, x)$ (for instance, the temperature along a rod, at each time) is the recording. The instruments are a few *basic behaviours*:

- **moving**: a shape travels without changing (transport);
- **spreading**: a sharp shape becomes smooth (diffusion);
- **growing or fading**: the height changes in time (reaction, decay).

In the mathematics, each basic behaviour is called a *factor* of the equation. The method answers the question:

Idea

Given a recording u of an unknown system, *which factors are present, and how strongly?* Then: *do the factors really act separately, or are they tangled together?*

What goes in and what comes out

Input and output

Input: a dataset of measurements (t_i, x_i, y_i), where $y_i \approx u(t_i, x_i)$.

Output:

1. a *list of factors with weights* ("transport 70 %, diffusion 30 %") together with their uncertainty, i.e. a probability distribution over factors;
2. a number telling how *tangled* the factors are (small = clean separation);
3. a reconstructed differential equation, built from the most probable factors;
4. as a by-product, which parts of the data favour which factor.

The pipeline in five moves

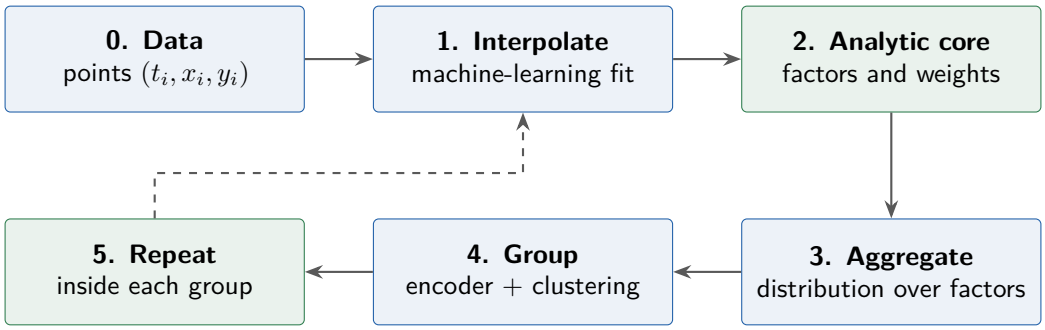


Figure 1: The pipeline. Green boxes are the exact mathematical parts (already built); blue boxes are the machine-learning parts (to be designed). The dashed arrow is the loop: each group becomes a new data set.

1. **Interpolate.** A learning algorithm fits a smooth function to the data. Because data are finite, it returns not one function but *many plausible ones, each with a probability.*

2. **Analytic core.** Each plausible function is handed to my algorithm. It spits out the factors that function contains and how much of the function each factor carries.

3. **Aggregate.** Add everything up, weighting by probability: we obtain a distribution ("transport appears with weight 0.7 ± 0.1, diffusion with 0.3 ± 0.1").

4. **Group.** Use this distribution to *set the parameters* of a standard grouping method (how many groups, how spread out) and cut the data into groups, so that each group ideally "sounds like one instrument".

5. **Repeat inside every group.** If the grouping was good, each group contains one (at most two) factors, and the tangle number is tiny. If not, regroup. When all groups are clean, the factors rebuild the equation.

A tiny example, start to finish

Suppose the (hidden) truth is

$$u(t,x) \;=\; 0.7\,e^{-(x-t)^2} \;+\; 0.3\,e^{-t}\cos x.$$

The first piece is a bump that *travels* to the right with speed 1. The second is a wave that *spreads and fades*. We only see noisy samples of u.

- **Step 1** returns many smooth curves close to u, each with a probability.
- **Step 2**, applied to each curve, finds the two factors

$$Q_1 = \partial_t + \partial_x \;\;\text{(transport)}, \qquad Q_2 = \partial_t - \partial_x^2 \;\;\text{(diffusion)},$$

  and the pieces $u_1 = 0.7\,e^{-(x-t)^2}$ and $u_2 = 0.3\,e^{-t}\cos x$.
- **Step 3** reports: transport $\approx 70\,\%$, diffusion $\approx 30\,\%$, with their spread.
- **Step 4** uses "two main factors" to choose "two groups" and cuts the data accordingly.
- **Step 5** re-analyses each group alone: the first group shows only transport, the second only diffusion, and the tangle number is close to zero in both. We stop and write down the equation.

Part II — A little mathematics

Scope

Everything in the analytic core assumes **exact** data and a **linear equation with constant coefficients**. This is the part I have worked out with complete proofs. The statistical wrapper (Sections 0.3–0.4) is a *proposal*: this is where I need you.

0.1 Setting

Let $u(t,x)$ be defined on a bounded region $\Omega \subset \mathbb{R}^2$. It satisfies a linear equation with constant coefficients $P(\partial_t, \partial_x)\,u = 0$ that is *not* known. Replacing $\partial_t \to \tau$, $\partial_x \to \xi$ turns an operator into a polynomial, its *symbol*. Polynomials factor, and the factors are our "instruments":

$$P = Q_1 Q_2 \cdots Q_N.$$

For the example above, $P = (\tau + \xi)(\tau - \xi^2)$.

0.2 The analytic core (exact data)

(i) Factors generate components. If the Q_j are pairwise coprime and Ω is simply connected, every solution splits as

$$u = \sum_{j=1}^{N} u_j, \qquad Q_j(\partial)\,u_j = 0.$$

The split is unique up to a finite-dimensional space of exponential-polynomials, of dimension

$$\delta(P) = \sum_{i<j} \dim_{\mathbb{C}}\, \mathbb{C}[\tau,\xi]/(Q_i, Q_j).$$

(This is the Chinese Remainder Theorem in two variables; the ambiguity is a few exponentials, not a function.) *Splitting by additive terms of P instead of factors is false in general.*

(ii) Evolution factors have an explicit projector. If $Q_j = \partial_t - a_j(\partial_x)$ with distinct a_j, then applying all the other factors kills every other component:

$$\prod_{k\neq j} (\partial_t - a_k(\partial_x))\, u \;=\; \prod_{k\neq j} (a_j - a_k)(\partial_x)\, u_j.$$

So u_j is recovered by "dividing" by $\prod_{k\neq j}(a_j - a_k)(\partial_x)$, up to its kernel (this is the finite ambiguity). In the example, $a_1 = -\partial_x$, $a_2 = \partial_x^2$, and the division is by $-\partial_x(1+\partial_x)$, whose kernel $\operatorname{span}\{1, e^{-x}\}$ has dimension $2 = \delta(P)$.

(iii) Terms inside a factor. A single factor $\partial_t - a(\partial_x)$ may itself be a sum of *terms*, $a = \sum_n a_n$ (advection, diffusion, ...). In Fourier variables $\hat{u}(t,\kappa) = \int u\, e^{-i\kappa x}\, \mathrm{d}x$,

$$\log \hat{u}(t,\kappa) \;=\; \log \hat{u}_0(\kappa) \;+\; t \sum_{n=1}^{N} a_n(\kappa),$$

which is *linear in the terms*: they are read off by regression on monomials in κ. For advection–diffusion $u_t = -cu_x + Du_{xx}$ one finds $a_1 = -ic\kappa$, $a_2 = -D\kappa^2$.

(iv) The interaction term. Let $S_n(t) = e^{tA_n}$ be the evolution generated by term n alone and $\Delta_n = S_n(t) - \mathrm{Id}$. Define the part of u that *no single term explains*:

$$E_{\mathrm{int}} \;=\; u(t) - u_0 - \sum_{n=1}^{N} (S_n(t)u_0 - u_0).$$

For commuting terms (constant coefficients) it has the exact closed form

$$E_{\mathrm{int}} = \Big[\prod_{n=1}^{N} (\mathrm{Id} + \Delta_n) - \mathrm{Id} - \sum_{n=1}^{N} \Delta_n\Big] u_0,$$

or, in Fourier variables,

$$\widehat{E_{\mathrm{int}}}(t,\kappa) = \Big[\prod_{n} e^{t a_n(\kappa)} - 1 - \sum_{n} (e^{t a_n(\kappa)} - 1)\Big] \hat{u}_0(\kappa).$$

Its value costs $O(N)$ operations (never expand the product). Two properties give the diagnostic its meaning: $E_{\mathrm{int}} = 0$ if only one term acts ($N = 1$), and for $N = 2$ we have $E_{\mathrm{int}} = \Delta_1\Delta_2 u_0$, the *product* of the two single-term effects. For advection–diffusion,

$$\frac{|\widehat{E_{\mathrm{int}}}|}{|\hat{u}_0|} = \left|(e^{-ic\kappa t} - 1)(e^{-D\kappa^2 t} - 1)\right| \leq 2.$$

Two different notions of "tangle"

It is useful to keep two quantities apart. **Between factors**: how hard is it to separate u_1 from u_2? For two factors this is measured by the Friedrichs cosine c_F (the cosine of the smallest angle between $\ker Q_1$ and $\ker Q_2$, modulo their intersection): $c_F \to 1$ means the components are nearly indistinguishable. **Inside a factor**: how much do its terms interact? This is E_{int}. A good grouping makes both small. A single factor with two real terms (advection–diffusion) is a legitimate output even though $E_{\mathrm{int}} \neq 0$: there E_{int} is information, not an error.

0.3 Statistical wrapper (proposal)

Step 1: a distribution of interpolants. The interpolation step returns a weighted family

$$\{(\hat{u}_s, \pi_s)\}_{s=1}^{S}, \qquad \pi_s \geq 0, \quad \sum_s \pi_s = 1,$$

for instance posterior samples from a Gaussian process, a Bayesian neural network, or an ensemble. The *basis functions* of this fit are the object of Question 1: they may be (a) solutions of known operators (e.g. $e^{\lambda t + i\kappa x}$, travelling waves), so each basis function maps directly to a factor, or (b) generic functions (networks, kernels), in which case the analytic core must extract factors from the fitted function itself.

Step 2: analytic core, sample by sample. For each $\hat{u}_s$ the core returns factors $Q_j^{(s)}$ (with coefficient vectors $\theta_j^{(s)}$ in the space of operator coefficients) and components $u_j^{(s)}$. Define the weight of a factor as its share of the signal,

$$w_j^{(s)} = \frac{\|u_j^{(s)}\|}{\sum_k \|u_k^{(s)}\|},$$

choosing the minimum-norm representative to fix the finite ambiguity. If $\hat{u}_s$ is a linear combination of basis functions, this is simply the (normalised) coefficient mass attached to each factor, as in the toy example, where a coefficient of 0.5 on a basis function sends weight 0.5 to its factor.

Step 3: distribution over factors. The result of aggregation is a weighted cloud in coefficient space,

$$\mathcal{W} \;=\; \sum_{s=1}^{S} \pi_s \sum_j w_j^{(s)} \; \delta_{\theta_j^{(s)}}.$$

From $\mathcal{W}$ we read: the dominant factors (modes of the cloud) with their total mass W_j, their means $\bar{\theta}_j$ and covariances Σ_j; the *effective number of factors*

$$k_{\text{eff}} = \exp\Big(-\sum_j W_j \log W_j \Big);$$

and the co-occurrence of factors across samples (which factors appear together), plus the average interaction and the average separability,

$$\bar{\rho} = \sum_s \pi_s \, \frac{\|E_{\text{int}}(\hat{u}_s)\|}{\|\hat{u}_s - u_0\|}, \qquad \bar{c}_F = \sum_s \pi_s \, c_F(\hat{u}_s).$$

Step 4: using $\mathcal{W}$ to set the clustering. My proposal (to be criticised): the number of groups is $k = \lceil k_{\text{eff}} \rceil$; the initial centres are the $\bar{\theta}_j$; the within-group variances are the Σ_j; the encoder dimension is of the order of the number of dominant factors, and the encoder is any of PCA, SVD, an autoencoder; the grouping algorithm is any of k-means, a Gaussian mixture, spectral clustering.

Two natural *notions of group* appear in the analytic theory, and one could use either, even different ones at different stages: (a) a group is a *kernel* $\ker Q_j$, i.e. functions annihilated by a factor ("multiplication by the operator" picture); (b) a group is a *flow family* $\{e^{tA_n} u_0\}$, i.e. what a term does alone ("exponential of the operator" picture).

0.4 The loop

Algorithm (outer loop)

1. Fit the dataset $\mathcal{D} \Rightarrow \{(\hat{u}_s, \pi_s)\}$.
2. Run the analytic core on every $\hat{u}_s \Rightarrow \mathcal{W}$ and the statistics k_{eff}, $\bar{\rho}$, $\bar{c}_F$.
3. If the groups are *clean* (each has ≤ 2 factors, $\bar{\rho} < \varepsilon_1$, $\bar{c}_F < \varepsilon_2$), **stop**.
4. Otherwise set the encoder and clustering parameters from $\mathcal{W}$, split $\mathcal{D}$ into $\mathcal{D}_1, \ldots, \mathcal{D}_k$, and **recurse** on each $\mathcal{D}_g$.
5. Reconstruct the equation from the surviving factors, with their probabilities.

0.5 What is known and what is not

Statement	Status	Remark
Factors give components; ambiguity $\delta(P)$	proved (exact data)	simply connected Ω; fails otherwise
Components are in H^m for all u	iff sum of kernels closed	needs a gap condition ($\gamma_N > 0$); not automatic
Closed formula for E_{int}, cost $O(N)$	proved	constant coefficients (commuting terms)
$u_0 \mapsto E_{\text{int}}$ continuous in H^m	**false**	stability under noise is a real issue
Finite data certify the equation	**impossible**	no finite certificate; black box impossible
Nonlinear extension	partial	Burgers: closed form before the shock
Dictionary identification, nonlinear	exact modulo differential consequences	a countable dictionary cannot identify everything
Statistical wrapper (II.3–II.4)	**proposal**	no convergence or noise-robustness theory

Table 1: What the analytic theory guarantees, and what is still open.

The last rows are the reason for this note: the analytic core is exact but fragile under noise, and the statistical wrapper is exactly the mechanism that could make it usable.

Part III — Questions for you

Q1 — Must the basis functions be chosen in advance?

If the interpolation uses basis functions that solve known equations (exponentials for constant coefficients, travelling waves for transport, Bessel or Airy functions for others), then the method is easy to implement, but *it can only discover equations whose building blocks are in the list.* For example, if there is no transport-type basis function, a transport equation may be misidentified. A fully generic fit (neural network, kernel method, Gaussian process) avoids the choice, but then the analytic core must extract factors from a function with no built-in structure.

Is there a standard way to avoid committing to a basis in advance (e.g. learning the basis, a nonparametric fit followed by Fourier- or jet-based extraction, a hierarchy of dictionaries), **and what does one lose in doing so?**

Relevant fact from the analytic side: no countable dictionary identifies everything exactly, and with a fixed dictionary one only recovers an equation modulo its differential

consequences. So the question is not purely technical.

Q2 — Designing the outer structure

I constructed the analytic core. I would like advice on everything around it:

1. **Loop and stopping rule.** Is "recurse until each group is clean" sensible? How to choose the thresholds $\varepsilon_1, \varepsilon_2$?
2. **Parameter estimation.** Is deriving k, the centres and the variances from the factor distribution $\mathcal{W}$ reasonable? Would you instead learn them jointly (end-to-end, EM, variational), with the analytic core as a differentiable layer?
3. **Choice of encoder and clustering.** PCA, SVD, an autoencoder; k-means, Gaussian mixtures, spectral methods. Which are appropriate when the "clusters" are defined by a differential structure rather than by distance?
4. **Uncertainty.** How should the probability from the interpolation step be propagated through a non-linear, possibly discontinuous map (the interaction term is not continuous in H^m)?
5. **Noise and validation.** What would a convincing experiment look like (synthetic data from known equations first)? Which baselines (SINDy, PDE-FIND, neural PDE learners) should it be compared against?

Proposed way of working. Start with synthetic data from two or three known linear equations (transport + diffusion mixtures like the example above). I supply the analytic core as a routine, together with the written proofs of the properties above; you design the outer structure and the experiments. The first deliverable would be the reconstructed factor distribution on those synthetic cases, compared against known ground truth.